From Probabilities to Decisions: Search and Multi-Teacher Distillation with Jev

Mohamad Yazan Sadoun Sarah Sharif Yaser Mike Banad
INQUIRE Lab, School of Electrical and Computer Engineering
University of Oklahoma, Norman, OK, USA

arXiv:2610.09188v1 [cs.AI] 6 Oct 2026

Abstract

Probability-only models, which TypeSafe calls System One models, return calibrated probabilities for fixed choices in milliseconds and generate no text. We study one such model, Jev, through two tasks that require decisions under tight constraints. In bullet chess, a bot that places Jev's judgment inside Stockfish search alongside an opening book and endgame tablebases climbs above a 2200 Lichess bullet rating against other bots. Live model calls are too slow for search, so we distill pairwise judgments into a compact evaluator that runs at every position. We then ask how best to spend a fixed labeling budget when an LLM, Qwen3-32B, is available as a second teacher. In chess, averaging both judges' labels beats spending the whole budget on Qwen alone by 9.6 Elo (95% interval 4.3 to 14.9), and the gain replicates on fresh openings; a second answer from the same judge is no substitute, and Jev is the strongest partner for Qwen among the models tested. In passage reranking, Jev's labels alone train a reranker that scores as high as Qwen's, from 21 minutes of API calls instead of 5.1 GPU-hours, and adding Qwen gains at most a few thousandths in ranking quality. Search supplies the lookahead, distillation makes the judgment cheap enough to use at every position, and an LLM partner pays off in chess.

1 Introduction

A system that acts under time pressure needs to compare alternatives, anticipate their consequences, and choose before its time runs out. A probability-only model offers one part of that process: a judgment about specified choices. TypeSafe's System One model, Jev, returns probabilities without generating text [1]. The name echoes the fast, intuitive System 1 of dual-process accounts of human thinking [2]. Jev's interface raises a direct question: how much can a decision system do with these judgments?

We investigate that question by connecting Jev to tools that supply what a fixed-choice answer leaves to the surrounding system. Game-tree search explores future consequences, stored opening and endgame knowledge handles familiar situations, and a learned evaluator scores the positions search reaches. Bullet chess is a demanding proof of concept because long-term consequences must be considered within a short clock. The reasoning process we study is this composition of search, knowledge, and evaluation.

The practical obstacle is repeated access to the model. Even a fast remote judgment is too expensive to request at every position in a search tree. We therefore collect preferences offline and distill them into a compact evaluator. This also gives us a place to combine Jev with an LLM: average their probabilities before fitting the evaluator, then use the evaluator throughout search. The question becomes whether the second judge improves the decisions made by the complete system for the same labeling budget.

The paper follows this progression from a deployed bot to controlled tests of the learned evaluator and its LLM partnership. The chess experiments show a benefit from combining Jev with Qwen, including against a single judge allowed twice as many training pairs. Passage reranking tests the broader use of probability-only labels with a different model and task. There, Jev alone trains a useful reranker, and adding Qwen gains at most $+0.004$. Together, the two tasks show that distilled probability-only judgments become reusable evaluators, and that an LLM partner's value depends on the task.

2 Putting Jev's Judgment to Work

2.1 Chess as a proof of concept

Our initial bot connects Jev-derived evaluation to Stockfish search [3]. Search follows candidate moves and replies; material values account for the pieces on the board, and a 411-weight evaluator trained from Jev supplies positional preferences. A masters opening book supplies established continuations, while Syzygy tablebases supply exact information in covered endgames [4]. The bot also queries Jev for choices among near-equal moves. These components put model judgment inside a system that can look ahead and use stored chess knowledge.

The bot's recorded Lichess bullet trajectory rises from 1753 to 2268 (Figure 1; Appendix C). This is a result for the complete bot, with all of those components available. To measure what learned judgment contributes, the subsequent experiments use a common search engine and disable the opening book, tablebases, and live queries during play. They also use the expanded 443-weight evaluator defined below.

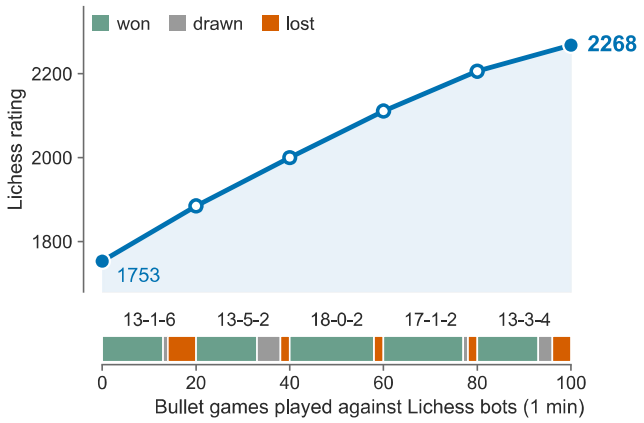


Figure 1: The complete bot's Lichess bullet trajectory at one minute per side. The strip groups wins (green), draws (grey), and losses (orange) by deployment round.

2.2 Distilling judgment for fast search

The deployment exposes a mismatch between model access and search. A live Jev call takes 177 ms at the median, while search evaluates thousands of positions per move. In a controlled test, even querying Jev only to break exact ties among the search's best moves reduces strength by 28.7 Elo (normal 95% interval $[-51.1, -6.6]$) over 400 games and 3,875 calls. A useful judgment must reach the search at a cost the clock can support.

Distillation moves that cost to training. We ask the model to compare moves offline, fit a small evaluator to its probabilities, and store the resulting weights in the engine. Search can then use what the evaluator learned at every evaluated position without waiting for a model response. The same training step can absorb an LLM's judgments, so a partnership need not add any model calls during play.

3 Distilling One or More Judges

A shared judgment interface. Let s be a position and a, b two candidate moves. Each judge answers "Which of these two moves is better for the side to move?" in both presentation orders; we map the answers to $p(a \succ b)$ and average them. Jev supplies the probabilities directly for the specified choices [1]. Qwen3-32B [5] receives the same board text and question, with reasoning mode off; we normalize the probabilities it assigns to the two move answers. OLMo-2-0325-32B-Instruct [6] follows the same procedure. DiffusionGemma-26B [7] runs through djev [8], a service that returns probabilities for specified choices.

Combining labels. For the Jev–Qwen partnership, the training target is the mean

$$p_F(a,b) = \tfrac{1}{2}\big(p_J(a,b) + p_Q(a,b)\big). \tag{1}$$

Single-judge evaluators use that judge's probabilities; other partnerships use the corresponding average, a form of multi-teacher distillation [9, 10]. This combination isolates whether a second judge's answers improve the evaluator.

A compact evaluator. Let $x(s,a)$ describe the position after move a, from the mover's perspective. Writing $x(s)$ for the features of position s, the evaluator computes a weighted sum $t_w(s) = w^\top x(s)$, fitted by

$$\min_w \sum_{(s,a,b)} \ell\Big(p(a,b), \sigma(w^\top \Delta x)\Big) + \frac{\lambda}{2}\|w\|_2^2, \tag{2}$$

where $\Delta x = x(s,a) - x(s,b)$ and ℓ is binary cross-entropy against the judge's probability. This is a Bradley–Terry preference model [11]; λ controls the penalty on weight size. The evaluator has 443 weights: 384 piece-square weights and 59 for mobility, pawn structure, king attacks, threats, and other positional features. We centre each piece's square weights at zero to preserve fixed material values.

Using the evaluator in search. The engine evaluates

$$E(s) = m(s) + H(s) + \alpha\, t_w(s), \tag{3}$$

where m scores material, H is a hand-written positional baseline, and α sets the learned score's scale. Before play, we fold the baseline and learned terms into stored integer weights. Stockfish 17.1 uses this evaluator in place of its neural-network evaluation; all evaluators run in the same executable with their own weight files. Correction history is disabled so that leaf scores come from the evaluator alone. The deployed evaluator requires neither judge (Figure 2a).

4 Controlled Evaluation

The controlled study changes the learned judgment while holding the search and evaluator form fixed. It measures whether a label source improves play, whether a partnership is worth its additional answers, and whether that result survives changes to the labels or partner.

Training pairs and response budgets. Four judges label 63,135 candidate pairs drawn from 13,000 positions in 2,042 source games. Candidates are the up-to-four best moves of a material-only Stockfish search (MultiPV 4, 0.1 s per position, one thread) within 20 centipawns of its top line. We ask every unordered pair in both presentation orders. The prompt lists occupied squares, the side to move, and the two moves (Appendix M). Each judge supplies 126,270 probabilities, so Jev+Qwen uses 252,540 responses. For an equal-budget comparison, Jev and Qwen each label a second block of 63,135 pairs on new positions. Its evaluator then receives the same number of responses as the combined evaluator.

Fitting and baselines. We split by source game, holding out one fifth for evaluation and using a further split of the rest to select λ by validation log loss. Every judge uses the same code, features, split rule, and candidate penalties (Appendix A). The weaker baseline, H_m, counts knight, bishop, rook, and queen moves, each weighted 1.0. The stronger baseline, H_s, adds the Simplified Evaluation Function's piece-square values and material adjustments [12]. It beats H_m by 55.2 Elo [29.6, 81.4]. Each

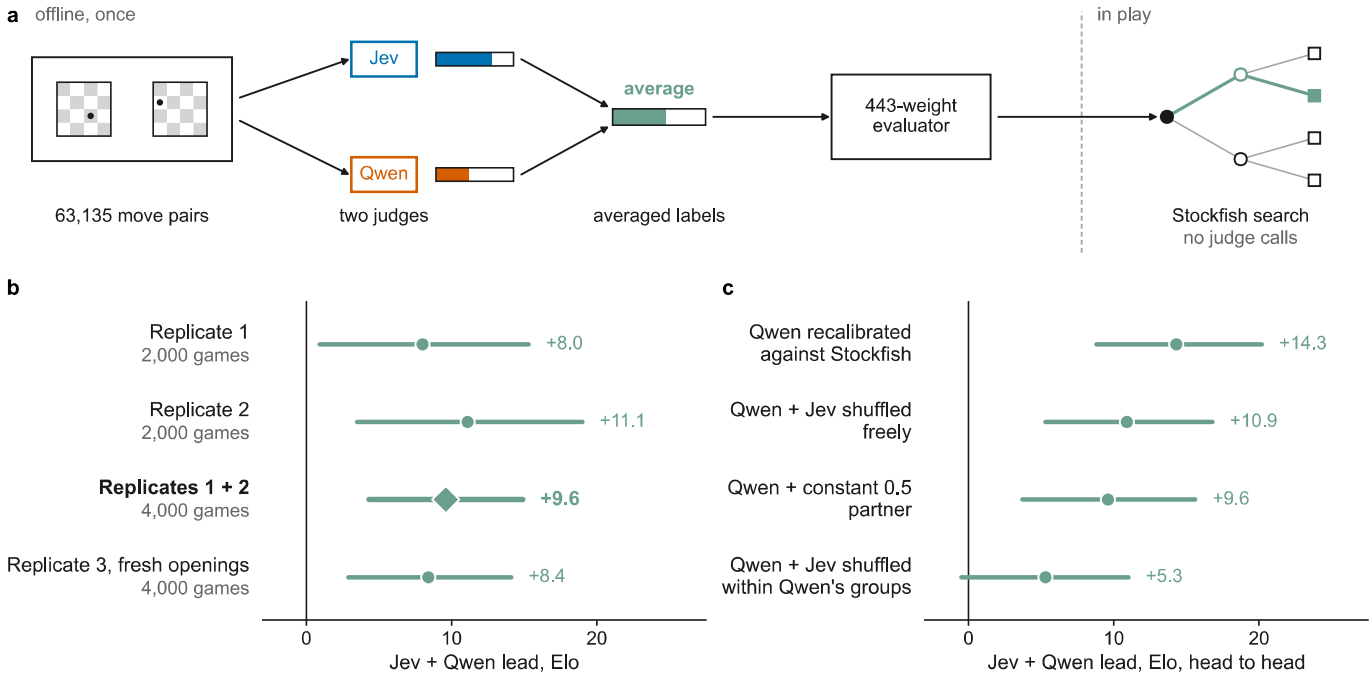


Figure 2: (a) Distilling judge probabilities into search weights. (b) Combined evaluator vs. Qwen on twice the pairs at equal budget; 2,000, 2,000, 4,000 (pooled), and 4,000 games. (c) Substitute controls, 4,000 games each. Dots: estimates; lines: 95% intervals.

comparison holds the baseline fixed and changes the learned addition.

Scale selection. The learned score must work at the scale expected by the search. Adding it to H_s at full scale reduces playing strength; rescaling H_s alone costs -49.0 Elo $[-72.2, -26.1]$. We set each learned score's standard deviation to $c \in \{0.125, 0.25, 0.5, 1\}$ times that of H_s, play 200 validation games per setting against H_s, and select the best, preferring the smaller value in a tie. All evaluators receive the same calibration budget and validation openings.

Playing strength. Games use 60 seconds per side, no increment, one thread, and a 16 MB position cache; each engine manages its own time. We play every opening with both colours and count draws as half a point. For a fraction p of points scored, the Elo difference is $400 \log_{10}(p/(1-p))$. Unless otherwise stated, chess-match intervals are 95% percentile intervals from 10,000 bootstrap resamples of opening clusters, keeping both colours together. Table 1 row 2 clusters by single opening (1,000 groups), rows 1 and 4 by a shared 20-ply book prefix (556 groups), and rows 3 and 5 by an 8-ply prefix (411 groups). Opponents in a comparison use the same openings, while scale selection has a reserved set and new comparisons draw fresh sets. Appendix L gives the full protocol.

Search-generated labels. A reference evaluator learns Stockfish 17.1's move preferences or position values in centipawns. It tests what the same features and fitting procedure can capture from labels generated by a chess search.

5 What the LLM Partnership Adds

5.1 Learning a useful search evaluator

Jev's labels contain information that a small evaluator can use in search. On the mobility baseline, the Jev-trained evaluator adds 38.9 Elo [29.9, 47.9] over 2,000 games. Combining it with Qwen's labels gives a larger improvement: in direct 400-game matches, Jev+Qwen beats Jev alone by 77.7 Elo [52.5, 103.7], Qwen alone by 54.3 Elo [32.2, 77.7], and a mobility score rescaled to the same size by 84.1 Elo [60.5, 109.5].

The stronger baseline leaves less room for Jev alone. The gains over H_s are $+2.8$ $[-5.5, 11.0]$ for Jev, $+16.7$ $[9.0, 24.4]$ for Qwen, and $+33.8$ $[25.2, 42.4]$ for their combination. The combined evaluator beats Qwen's in a direct 2,000-game match by 18.1 Elo [10.8, 25.5] (640 wins, 824 draws, 536 losses). A label source's value therefore depends on what the baseline and the other judge already supply. Against Stockfish limited to Elo 2,400, the combined and published-table evaluators score $+376.9$ [336.1, 424.3] and $+334.1$ [296.3, 376.9], respectively, in 400 games each on the same 200 openings.

5.2 A partnership or more training pairs?

A second judge competes with another use of the budget: more examples from the first judge. We train Jev-only and Qwen-only evaluators on twice the pairs, giving each the same 252,540 responses and scale-selection budget as Jev+Qwen. On 1,000 fresh openings played with both colours, the combination beats both (Table 1). Pooling two full-data matches against Qwen on twice the pairs gives $+9.6$ Elo [4.3, 14.9] over 4,000 games; the individual estimates are $+8.0$ and $+11.1$. A further 4,000-game

Table 1: Jev+Qwen vs. each rival at 252,540 responses, 2,000 games. Row 2 clusters by single opening; rows 1 and 4 by 20-ply prefix; rows 3 and 5 by 8-ply.

Rival (252,540 responses each)	Elo	95% interval
Jev, twice the pairs	+27.0	[19.2, 34.8]
Qwen, twice the pairs	+8.0	[0.9, 15.3]
Qwen, two answers averaged	+14.3	[5.9, 22.5]
Jev, two answers averaged	+28.2	[20.2, 36.3]
Qwen, adjusted toward 0.5 (τ=0.4465)	+8.0	[0.5, 15.4]

Table 2: Jev+Qwen vs. four substitutes, 4,000 games on fresh openings at the same response budget.

Substitute for Jev	Elo	95% interval
Jev replaced by 0.5	+9.6	[3.7, 15.6]
Jev probabilities shuffled freely	+10.9	[5.3, 16.8]
Qwen alone, recalibrated, twice the pairs	+14.3	[8.8, 20.2]
Jev shuffled within Qwen groups	+5.3	[−0.5, 11.0]

match on fresh openings gives +8.4 [2.9, 14.1] (1,152 wins, 1,793 draws, 1,055 losses).

Repeated answers from one judge provide another equal-budget control. Averaging two Jev answers, which agree on 92.1% of pairs, or two Qwen answers still produces a weaker evaluator than Jev+Qwen. Adjusting Qwen's probabilities to match the combination's average distance from 0.5 (τ=0.4465) also leaves a gap (Table 1). In these comparisons, neither extra pairs nor repeated answers from one judge recover the benefit of the partnership.

5.3 The information in Jev's labels

Averaging probabilities changes both their values and their certainty. We test whether those general changes explain the improvement by replacing Jev's contribution while keeping the response budget fixed (Table 2; Figure 2c). Each comparison uses 4,000 games on fresh openings.

Replacing Jev by 0.5 softens every Qwen label equally; freely shuffling Jev's probabilities keeps their distribution but breaks their connection to the position. Both substitutions weaken the evaluator. Recalibrating Qwen against Stockfish with a fitted temperature $T \approx 18$ and using twice the pairs also trails the real combination. These controls support a contribution from Jev's position-dependent judgments beyond uniform softening or this recalibration of Qwen.

A stricter shuffle moves Jev's probabilities only within ten equal-sized groups defined by Qwen's probability. It preserves their association at that group level. The combination's lead over this stricter substitute is +5.3 Elo [−0.5, 11.0]. The free shuffle shows that the match between Jev's probabilities and their positions carries the gain.

5.4 Choosing a partner

We next ask whether another model provides the same benefit alongside Qwen. Each evaluator is fitted on three overlapping subsets, each containing two of three groups of source games

Table 3: Jev+Qwen's lead over alternative Qwen partnerships, 6,000 games per evaluator across three subsets. Intervals resample openings.

Jev+Qwen's lead over	Elo	95% interval
Full-data settings		
OLMo+Qwen	+16.6	[10.3, 22.8]
DiffusionGemma+Qwen	+16.9	[10.2, 23.4]
Settings selected per subset		
OLMo+Qwen	+7.7	[1.4, 14.0]
DiffusionGemma+Qwen	+7.9	[1.3, 14.6]

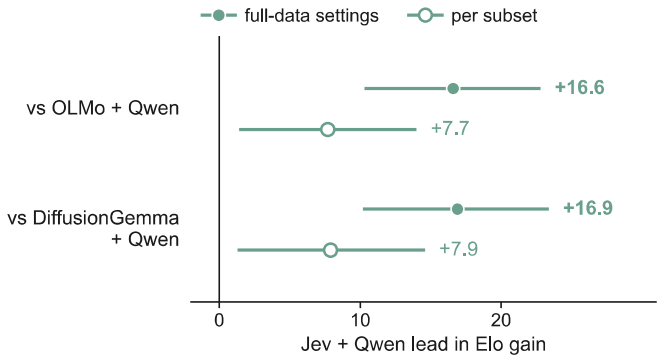


Figure 3: Jev+Qwen's lead over alternative partnerships, 6,000 games per evaluator. Filled: full-data settings; open: per-subset. Lines: 95% intervals.

and about half the pairs. Within each run, all evaluators play the corresponding Qwen evaluator on the same 1,000 colour-paired openings. We compare two rules for λ: retain the full-data setting (300 for the combinations, 100 for Qwen on twice the pairs), or select it anew within each subset.

Jev+Qwen leads OLMo+Qwen and DiffusionGemma+Qwen under both rules (Table 3). The lead is smaller when each subset selects its own penalty. Under that rule, Jev+Qwen's advantage over Qwen on twice the pairs is +2.2 Elo [−2.4, 6.8]. The full-data result is +9.6 Elo [4.3, 14.9] (Section 5.2). Figure 3 and Appendices I–J show the effect of these training choices. Testing all combinations of the four judges likewise gives no simple benefit from adding more judges (Appendix D).

5.5 Why measure decisions after search?

A label-level score does not explain the ordering of the resulting players. On 12,280 held-out pairs with a non-tied Stockfish preference, agreement is similar across judges: 54.5% for Jev, 52.9% for Qwen, 53.4% for OLMo, and 54.4% for DiffusionGemma. Jev has the lowest expected calibration error against those preferences, 0.081, compared with 0.159 for DiffusionGemma, 0.176 for OLMo, and 0.260 for Qwen.

Jev and Qwen disagree on 30.9% of the pairs (3,791 comparisons). On those disagreements, Jev matches Stockfish 53.6% of the time (95% interval [51.5, 55.6], resampling source games). The averaged probabilities match only 48.2%, yet they train the stronger full-data player. The same mismatch appears after fitting: on 1,285 quiet positions, Jev's evaluator has higher move

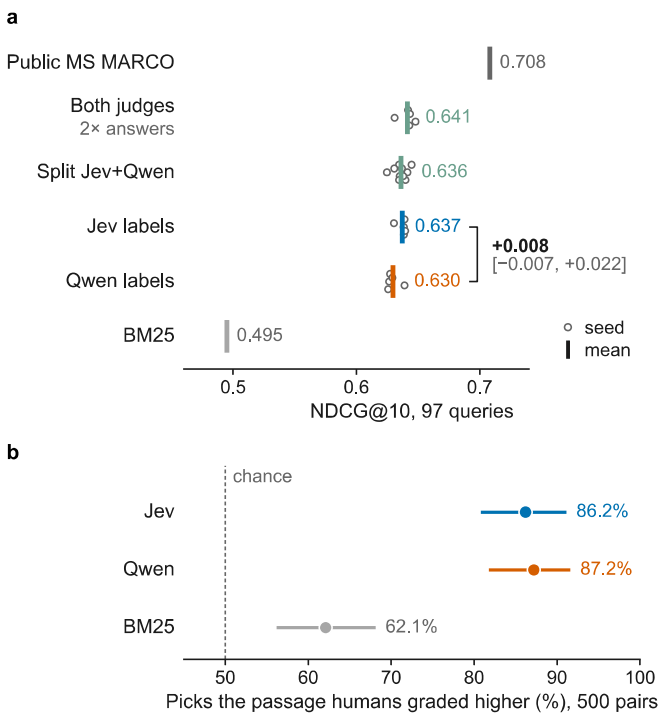


Figure 4: (a) Passage-reranking NDCG@10 on TREC DL 2019+2020. Open dots: individual training runs; tick: mean over five seeds, with both order assignments pooled for the split row. (b) Agreement with human graders on 500 pairs.

accuracy than the combined evaluator, 62.4% versus 60.7%, while losing in direct play. Appendix H gives the paired tests and a similar comparison using evaluators trained on Stockfish.

The evaluator supplies a score repeatedly within a search tree, where the final action depends on many evaluated continuations. The playing-strength test measures what static accuracy does not: how the score performs across a full search tree.

6 Beyond Chess: Passage Reranking

The labels idea extends beyond move evaluation. For text search, the choices are passages for a query, the reranker is a MiniLM reranker [13], and success is agreement with human relevance judgments. This provides a second test of what can be built from probability-only answers and whether an LLM partner improves it.

On 500 human-graded TREC Deep Learning 2019 passage pairs [14, 15], Jev chooses the more relevant passage 86.2% of the time and Qwen 87.2%, compared with 62.1% for BM25. We then collect both judges' probabilities on 80,000 MS MARCO pairs [16], in both presentation orders, and train rerankers for 10,000 steps with five seeds. Each reranker reranks BM25's top 100 passages for 97 human-judged TREC Deep Learning 2019 and 2020 queries. It ranks independently after training, with no judge calls.

Useful labels from Jev alone. Jev's reranker scores 0.637 in NDCG@10, a ranking-quality metric where higher is better; Qwen's scores 0.630 (unrounded, 0.6371 and 0.6295). Their difference is $+0.008$ $[-0.007, 0.022]$ with seeds and queries resampled: the two judges finish within 0.008 of each other. Jev's reranker improves on BM25's 0.495 by $+0.142$ $[0.098, 0.186]$. A public MS MARCO cross-encoder of the same size scores 0.708. Collecting Jev's 160,000 answers took about 21 minutes of API time with no local GPU; Qwen required about 5.1 GPU-hours in our labeling setup.

Spending the budget on coverage or another judge. Adding Qwen to Jev gains at most $+0.004$ $[-0.003, 0.011]$ in any tested combination. Splitting the two presentation orders between Jev and Qwen gives similar quality at the same answer budget. Using both judges on every pair doubles the responses and adds $+0.004$ $[-0.003, 0.011]$, with queries resampled. When the budget instead forces a choice between two judges on half the pairs and Jev on all the pairs, broader coverage wins: the combination loses 0.032 in NDCG@10 (95% interval $[0.018, 0.045]$) under matched 5,000-step training. Asking Qwen only about Jev's least certain pairs loses to Jev alone at 80,000 answers and does not beat it at 160,000. Appendix N reports these allocations and their intervals.

Chess and retrieval support the same approach but lead to different judge choices. The chess evaluator gains from the partnership at full data; the reranker trains as well from Jev alone, and adding Qwen gains at most $+0.004$.

7 What the Proof of Concept Shows

Reasoning through the surrounding system. The chess system obtains lookahead from search and positional guidance from learned judgments. Distillation makes that guidance cheap enough to reuse throughout the tree, and the deployed bot also draws on opening and endgame tools. The reasoning is the system's explicit search over continuations, guided by the learned evaluator. This separation of roles makes a probability-only interface useful even when the task requires considering consequences beyond the immediate choice.

Useful judgment is task dependent. A search can repeatedly use an unhelpful score as readily as a useful one. Jev-derived evaluation improves play in chess and Breakthrough but weakens it in Othello and Connect-4 (Appendix G). The Othello checks expose a concrete problem: the judges rarely prefer corner moves in the tested comparisons, and the Jev-trained evaluator assigns corners a negative weight (Appendix K). The labels need to contain knowledge suited to the task.

A partnership is a choice about labels. The chess controls show that Jev's contribution survives comparisons with repeated answers, extra pairs, and several substitutes. In chess, combining Jev and Qwen produces a stronger player; in passage reranking, Jev alone scores as high as Qwen and adding Qwen gains at most $+0.004$. Spend responses on another judge only when its labels improve the evaluator enough to justify the cost. Pairwise probabilities can support both a small search evaluator

Table 4: Cost of obtaining 126,270 judge responses (63,135 pairs, both orders). Effective rates reflect concurrent requests.

Judge	ms/resp	Timing	Hardware	Cost
Jev	7.6	20 concurrent	API; no local GPU	$1.97
Qwen3-32B	54.6	sequential	1× H200	1.91 GPU-h
OLMo-2-32B	35.6	sequential	PRO 6000 + H200	1.25 GPU-h
DiffusionGemma	7.9	32 concurrent	1× RTX 5090	0.28 GPU-h

and a text reranker, with the model chosen for the decisions it will make.

8 Labeling Cost and Reproduction

Moving judgment offline separates the cost of collecting labels from the cost of making a decision. Table 4 reports the measured configurations for 126,270 chess responses, covering 63,135 pairs in both orders. Jev cost $1.97 and about 16 minutes of API time at TypeSafe's stated $0.042 per million input tokens, with output free, 372.3 billed tokens per call, and no local GPU. Qwen and OLMo each used over an hour of GPU time while answering sequentially. DiffusionGemma, served through djev on one RTX 5090, completed the labels in 0.28 GPU-hours. Jev's row is API wall-clock time; the others are GPU time on the listed hardware. After fitting, all chess evaluators use the same 443-weight form during play. Appendices A, L, M, and N specify the splits, features, prompts, and training procedures.

9 Related Work

Distillation and learned feedback. Hinton et al. train a smaller network to match averaged output probabilities [9], and You et al. study learning from multiple judges [10]. Motif uses collected language-model preferences to learn a reward for reinforcement learning [17]. We use probability averaging and preference fitting as established tools, and study what a probability-only judge contributes when its evaluator is used in a decision system. The comparisons hold model size, search, and response budget fixed.

Combining imperfect judgments. Dawid and Skene model annotator error rates [18], while Snorkel combines imperfect label sources [19]. WARM averages reward-model weights to reduce exploitation of the learned reward [20]. Our evaluators learn from a simple average of probabilities, allowing the experiments to focus on the value of the additional judge and its labels.

LLMs and probability-only models as judges. Zheng et al. evaluate language models as judges of human preferences [21]; Verga et al. combine diverse smaller judges in panels [22]. Rao and Callison-Burch compare Jev and LLMs on rubric judgments and identify correlated failures [23]. Li et al. accept confident Jev judgments and send uncertain cases to a reasoning judge at query time [24]. Our setting transfers judgments into an evaluator that runs without either judge. The retrieval gate tests an offline version of uncertainty-based allocation.

Evaluation and planning in games. TDLeaf(λ) learns evaluation weights from search leaves [25], and Texel tuning fits them to game outcomes [26]. DeepChess learns pairwise comparisons for alpha-beta search [27]; SentiMATE learns move evaluations from human commentary [28]. NNUE embeds an efficiently updatable neural evaluator in traditional search [29]. Ruoss et al. train a transformer on Stockfish scores to play without search [30], while Schultz et al. study external and internal planning with language models in board games [31]. Our proof of concept distills fixed-choice model judgments into a compact evaluator and lets search supply the lookahead.

10 Conclusion

Probability-only judgments are useful components of systems that plan and act under time constraints. The chess proof of concept connects Jev-derived evaluation to search and supporting tools, then distills judgments into weights that search can reuse without live model calls. Combining Jev and Qwen improves the full-data evaluator even when either judge alone receives the same response budget. In passage reranking, Jev alone trains a useful reranker, and adding Qwen gains at most $+0.004$ in NDCG@10. Each component has a concrete role: search considers consequences, distillation makes judgment reusable, and an additional judge earns its budget by what it contributes to the deployed evaluator.

References

[1] TypeSafe AI. System One (concepts). `https://docs.typesafe.ai/concepts/system-one`, 2026. Documentation; accessed 2026-09-26.

[2] Daniel Kahneman. *Thinking, Fast and Slow*. Farrar, Straus and Giroux, New York, 2011.

[3] The Stockfish developers. Stockfish 17.1. `https://github.com/official-stockfish/Stockfish`, 2025. Source tag sf_17.1.

[4] Ronald de Man. Syzygy Tablebases. `https://github.com/syzygy1/tb`, 2013. Endgame tablebases; accessed 2026-09-30.

[5] Qwen Team. Qwen3-32B Model Card. `https://huggingface.co/Qwen/Qwen3-32B`, 2025. Accessed 2026-09-26.

[6] Allen Institute for AI. OLMo-2-0325-32B-Instruct Model Card. `https://huggingface.co/allenai/OLMo-2-0325-32B-Instruct`, 2025. Accessed 2026-09-30.

[7] Google. DiffusionGemma-26B-A4B. `https://huggingface.co/google/DiffusionGemma-26B-A4B`, 2026. Accessed 2026-09-30.

[8] Matt Mastracci. djev: Diffusion model judge endpoint. `https://github.com/mmastrac/djev`, 2026. Serves DiffusionGemma as a typed-choice endpoint; accessed 2026-09-30.

[9] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the Knowledge in a Neural Network. arXiv:1503.02531, 2015.

[10] Shan You, Chang Xu, Chao Xu, and Dacheng Tao. Learning from Multiple Teacher Networks. In *Proceedings of the 23rd ACM*

SIGKDD International Conference on Knowledge Discovery and Data Mining, pages 1285–1294, 2017. doi: 10.1145/3097983.3098135.

[11] Ralph Allan Bradley and Milton E. Terry. Rank analysis of incomplete block designs: I. The method of paired comparisons. *Biometrika*, 39(3/4):324–345, 1952. doi: 10.2307/2334029.

[12] Tomasz Michniewski. Simplified Evaluation Function. Chess Programming Wiki, `https://chessprogramming.org/Simplified_Evaluation_Function`, n.d. Published tables reproduced from the author's mailing-list contribution; accessed 2026-09-26.

[13] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers. In *Advances in Neural Information Processing Systems*, 2020. arXiv:2002.10957.

[14] Nick Craswell, Bhaskar Mitra, Emine Yilmaz, Daniel Campos, and Ellen M. Voorhees. Overview of the TREC 2019 Deep Learning Track. In *Proceedings of the Twenty-Eighth Text REtrieval Conference (TREC 2019)*, 2020. arXiv:2003.07820.

[15] Nick Craswell, Bhaskar Mitra, Emine Yilmaz, and Daniel Campos. Overview of the TREC 2020 Deep Learning Track. arXiv:2102.07662, 2021.

[16] Payal Bajaj, Daniel Campos, Nick Craswell, Li Deng, Jianfeng Gao, Xiaodong Liu, Rangan Majumder, Andrew McNamara, Bhaskar Mitra, Tri Nguyen, Mir Rosenberg, Xia Song, Alina Stoica, Saurabh Tiwary, and Tong Wang. MS MARCO: A Human Generated MAchine Reading COmprehension Dataset. In *Proceedings of the Workshop on Cognitive Computation*, 2018. arXiv:1611.09268.

[17] Martin Klissarov, Pierluca D'Oro, Shagun Sodhani, Roberta Raileanu, Pierre-Luc Bacon, Pascal Vincent, Amy Zhang, and Mikael Henaff. Motif: Intrinsic Motivation from Artificial Intelligence Feedback. arXiv:2310.00166, 2023.

[18] A. P. Dawid and A. M. Skene. Maximum Likelihood Estimation of Observer Error-Rates Using the EM Algorithm. *Journal of the Royal Statistical Society. Series C (Applied Statistics)*, 28(1): 20–28, 1979. doi: 10.2307/2346806.

[19] Alexander Ratner, Stephen H. Bach, Henry Ehrenberg, Jason Fries, Sen Wu, and Christopher Ré. Snorkel: Rapid Training Data Creation with Weak Supervision. *Proceedings of the VLDB Endowment*, 11(3):269–282, 2017. doi: 10.14778/3157794.3157797.

[20] Alexandre Ramé, Nino Vieillard, Léonard Hussenot, Robert Dadashi, Geoffrey Cideron, Olivier Bachem, and Johan Ferret. WARM: On the Benefits of Weight Averaged Reward Models. arXiv:2401.12187, 2024.

[21] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems*, 2023. arXiv:2306.05685.

[22] Pat Verga, Sebastian Hofstätter, Sophia Althammer, Yixuan Su, Aleksandra Piktus, Arkady Arkhangorodsky, Minjie Xu, Naomi White, and Patrick Lewis. Replacing Judges with Juries: Evaluating LLM Generations with a Panel of Diverse Models. arXiv:2404.18796, 2024.

[23] Delip Rao and Chris Callison-Burch. JEV vs. LLMs as Rubric Judges: Cheaper, Faster, and Wrong in the Same Places. arXiv:2609.29769, 2026.

[24] Yubo Li, Yidi Miao, Ramayya Krishnan, and Rema Padman. JEV-as-a-Judge: Accept When Confident, Escalate When Unsure. arXiv:2609.26550, 2026.

[25] Jonathan Baxter, Andrew Tridgell, and Lex Weaver. TDLeaf(λ): Combining temporal difference learning with game-tree search. arXiv:cs/9901001, 1999.

[26] Peter Osterlund. The Texel Evaluation Function Optimization Algorithm. `https://talkchess.com/forum/viewtopic.php?p=555522`, 2014. January 31.

[27] Eli (Omid) David, Nathan S. Netanyahu, and Lior Wolf. DeepChess: End-to-End Deep Neural Network for Automatic Learning in Chess. In *International Conference on Artificial Neural Networks*, volume 9887 of *Lecture Notes in Computer Science*, pages 88–96, 2016. URL `https://arxiv.org/abs/1711.09667`.

[28] Isaac Kamlish, Isaac Bentata Chocron, and Nicholas McCarthy. SentiMATE: Learning to play Chess through Natural Language Processing. arXiv:1907.08321, 2019.

[29] Yu Nasu. Efficiently Updatable Neural-Network-based Evaluation Functions for Computer Shogi. The 28th World Computer Shogi Championship Appeal Document, 2018. Translated by Dominik Klein; first applied to Stockfish 12 (2020).

[30] Anian Ruoss, Grégoire Delétang, Sourabh Medapati, Jordi Grau-Moya, Li Kevin Wenliang, Elliot Catt, John Reid, Cannada A. Lewis, Joel Veness, and Tim Genewein. Amortized Planning with Large-Scale Transformers: A Case Study on Chess. In *Advances in Neural Information Processing Systems*, 2024. arXiv:2402.04494.

[31] John Schultz, Jakub Adamek, Matej Jusup, Marc Lanctot, Michael Kaisers, Sarah Perrin, Daniel Hennes, Jeremy Shar, Cannada Lewis, Anian Ruoss, Tom Zahavy, Petar Veličković, Laurel Prince, Satinder Singh, Eric Malmi, and Nenad Tomašev. Mastering Board Games by External and Internal Planning with Language Models. In *Proceedings of the 42nd International Conference on Machine Learning*, 2025. arXiv:2412.12119.

A Fitting Details

We hold out one fifth of source games for evaluation (seed 8) and, within the rest, one fifth for selecting training settings (seed 9). Each feature is divided by its standard deviation in the training data; constant features are removed. The candidate values for the weight-size penalty λ are $\{1, 10, 100, 300, 1000, 3000, 10000, 30000\}$. Newton updates stop after 30 iterations or a maximum weight change below 10^{-8}. Exported weights are centred within each piece-square block, scaled to a standard deviation of one on 1,000 fixed positions, and rounded to integers with denominator 4096. The reference evaluator trained on Stockfish scores uses targets $\sigma(kv)$, where v is the position score in centipawns and $k = 0.00368208$. Material contributes a fixed offset in log odds.

Single-judge evaluators with the same response budget. The second training block contains the first 63,135 pairs from a fixed ordering of newly labelled positions. Jev and Qwen receive the same board format and question wording. The two blocks together draw on 2,145 source games, against 2,042 for the first block alone, so the second block adds new positions

from mostly the same source games. Jev answered the second block with all pairs of a position in one request; Qwen answered each item separately, as in the first block. Each single-judge evaluator trains on its labels from both blocks, totalling 126,270 pairs, using the same code as the main evaluators. Running this code on the original single-judge training sets reproduces their weight files exactly. The selected weight-size penalties are 10 for Jev and 100 for Qwen.

B Scale Selection

Table 6 lists every validation setting against the unchanged published tables, 200 games each on the same 100 colour-paired openings. Every later comparison uses the scale marked in bold.

Table 5 tests the single-judge evaluators trained on twice the pairs at alternative scales, using 200 unused openings.

Table 5: Jev+Qwen against single-judge evaluators trained on twice the pairs at the listed scales, 400 games on 200 unused openings.

Evaluator and scale c	Elo	95% interval
Jev, $c = 0.5$	$+32.2$	$[13.9, 50.7]$
Jev, $c = 0.125$	$+27.9$	$[10.4, 46.3]$
Qwen, $c = 0.5$	$+19.1$	$[4.3, 34.0]$
Qwen, $c = 0.125$	$+34.9$	$[16.5, 53.4]$

Table 6: Validation Elo against published tables. Each coefficient is the ratio of the learned evaluator's standard deviation to the baseline's. Bold marks the selected scale.

		Same pairs		Twice the pairs	
c	Jev+Qwen	Jev	Qwen	Jev	Qwen
0.125	$+33.1$	$+10.4$	$+\mathbf{24.4}$	$+10.4$	$+29.6$
0.25	$+31.4$	$+10.4$	0.0	$+\mathbf{24.4}$	$+\mathbf{34.9}$
0.5	$+\mathbf{57.9}$	$+\mathbf{12.2}$	$+17.4$	-3.5	$+17.4$
1.0	-5.2	-24.4	-125.0	-24.4	-68.6

C The Recorded Lichess Deployment

Figure 1 records a segment of the bot's deployment against other Lichess bots at one minute per side with no increment. The segment contains 74 wins, 10 draws, and 16 losses, with a rating rise from 1753 to 2268. The deployed bot combines Stockfish search, material count, a 411-weight Jev-trained evaluator, Syzygy endgame tablebases [4], a masters opening book, and live Jev choices among near-tied moves. The opening book supplies established continuations, tablebases supply exact information for covered endgames, and the evaluator guides search outside those resources. The controlled study uses the separate 443-weight evaluator and disables the book, tablebases, and live queries during play.

D Testing All Judge Combinations

To test whether more judges give a better evaluator, Table 7 covers all 15 nonempty combinations of the four judges. Each evaluator trains on the average of its judges' probabilities for 63,135 pairs, uses the same scale-selection procedure and plays 1,000 games against the published tables on the same 500 openings.

Table 7: All 15 judge combinations, 1,000 games each against the published-table evaluator on shared openings. n: number of judges.

Judges	n	Elo	95% interval
Jev+Qwen	2	$+25.8$	$[12.9, 38.4]$
Qwen	1	$+24.0$	$[13.2, 34.9]$
Jev+Qwen+DiffusionGemma	3	$+19.1$	$[8.0, 30.3]$
Qwen+OLMo+DiffusionGemma	3	$+13.6$	$[2.1, 25.1]$
Jev	1	$+13.2$	$[2.1, 24.0]$
OLMo+Qwen	2	$+13.2$	$[2.4, 24.7]$
All four	4	$+13.2$	$[2.1, 24.4]$
DiffusionGemma+Qwen	2	$+11.1$	$[0.7, 21.9]$
DiffusionGemma	1	$+10.4$	$[-0.3, 21.6]$
Jev+Qwen+OLMo	3	$+6.3$	$[-4.5, 17.4]$
OLMo	1	$+5.9$	$[-4.5, 16.3]$
OLMo+DiffusionGemma	2	$+4.5$	$[-6.6, 15.6]$
Jev+DiffusionGemma	2	-2.8	$[-13.2, 7.6]$
Jev+OLMo+DiffusionGemma	3	-3.5	$[-14.6, 8.0]$
Jev+OLMo	2	-5.9	$[-16.7, 4.9]$

E Locating the Learned Contribution

We swap groups of weights between the full-data Jev+Qwen and DiffusionGemma+Qwen evaluators to test whether one feature family explains the difference between these partnerships. Each modified evaluator replaces one group of weights, covering piece-square values, mobility, pawn structure, king safety, threats or other features, while retaining the original evaluator's scale. Swapping piece-square weights changes the learned evaluator's standard deviation to $0.33\times$ the original in one direction and $3.9\times$ in the other. Swapping any other group leaves it between $0.98\times$ and $1.13\times$ the original. Each modified evaluator plays 1,000 games against the published tables on a shared set of 500 openings (Table 8).

We also rescale the substituted piece-square weights to keep the learned evaluator's total standard deviation equal to the original evaluator's. With total score variation held fixed, swapping piece-square weights changes Jev+Qwen by -8.7 $[-25.8, 8.4]$ and DiffusionGemma+Qwen by $+9.7$ $[-5.2, 24.7]$.

F Results at Other Time Controls

At 180 seconds per side, Jev+Qwen leads Qwen trained on twice the pairs by 15.6 Elo $[5.7, 25.8]$ and Jev trained on twice the pairs by 17.4 Elo $[6.3, 28.7]$. At 10 seconds per side plus 0.1 seconds per move, the Elo differences are $+6.9$ $[-4.0, 18.1]$ against Qwen and $+21.9$ $[9.2, 34.7]$ against Jev. Figure 5b changes one judge at a time in an earlier single-subset comparison: Jev+Qwen leads OLMo+Qwen by $+6.3$ $[-1.4, 13.7]$, and

Table 8: Elo change from replacing one weight group, 1,000 games per row. Piece-square swaps change total score variation; rescaled swaps hold it fixed.

Weight group	ΔElo	95% interval
Jev+Qwen, replacing from DiffusionGemma+Qwen		
Piece-sq. (0.33× scale)	−7.7	[−23.8, 8.4]
Piece-sq. (matched scale)	−8.7	[−25.8, 8.4]
mobility	+3.1	[−13.6, 19.9]
king safety	−10.1	[−27.2, 6.6]
pawn structure	−7.3	[−24.1, 9.7]
threats	−10.5	[−28.6, 8.0]
other	−4.5	[−21.3, 12.6]
DiffusionGemma+Qwen, replacing from Jev+Qwen		
Piece-sq. (3.9× scale)	−4.5	[−19.8, 11.1]
Piece-sq. (matched scale)	+9.7	[−5.2, 24.7]
mobility	−0.7	[−15.6, 14.3]
king safety	+10.4	[−4.5, 25.4]
pawn structure	+1.0	[−14.3, 16.4]
threats	+13.9	[−0.3, 28.9]
other	+9.0	[−5.9, 24.4]

Jev+OLMo trails Qwen+OLMo by −12.2 [−19.7, −4.5]; Jev's partner advantage is measured with Qwen.

G Transfer to Other Games

The value of distilled judgment depends on the game. Adding a Jev-trained score improves chess and Breakthrough and weakens Othello and Connect-4. We apply the same comparison in three additional games: each game's hand-written evaluator, that evaluator plus a Jev-trained score, and a reference score trained from search-generated labels. Jev is the only judge whose labels train these scores (Figure 6). In chess, adding the Jev-trained score improves the mobility baseline by 38.9 Elo [29.9, 47.9] over 2,000 games. Training the same features on Stockfish preferences adds 206.2 Elo. Jev-trained scores reduce strength in Othello and Connect-4; the Connect-4 change is −157.7 Elo. Reference scores using the same features add +216.1 and +129.9 Elo, respectively. In Othello, subtracting the Jev-trained score improves play. On 5,856 corner-versus-non-corner pairs, Jev prefers the corner 14.4% of the time. All three tested judges prefer the non-corner move in most pairs (Qwen 17.7%, OLMo 15.3%; Jev on a coordinate-list rendering 24.3%; Appendix K). Jev's low preference for corner moves accompanies a loss of 619.4 Elo over 400 games when its trained score is added to the baseline. In Breakthrough, the Elo changes are +219.9 for the Jev-trained score and −52.5 for the reference score. Jev's gain is +25.1 above the random scores' 95th-percentile gain of +194.8; the random scores were tested on 50 of the 200 openings.

H Move Accuracy on Fixed Positions

On 1,285 quiet positions where Stockfish separates the two leading candidates by at least 80 centipawns, the Jev evaluator selects the better move 62.4% of the time and Jev+Qwen 60.7%. Jev has the higher move accuracy (exact McNemar p=0.15), while Jev+Qwen plays more strongly in direct games. Including material and mobility, move accuracy is 65.29% for Jev and 64.44% for Jev+Qwen. The Jev evaluator trained on twice the pairs selects 808 moves correctly against Jev+Qwen's 780 (p=0.07), while Jev+Qwen wins their direct match by 27.0 Elo.

Two Stockfish-trained evaluators using the same features also rank differently on move accuracy and playing strength (Table 9). On 12,280 held-out move comparisons, the evaluator trained on Stockfish position scores answers 58.1% correctly, compared with 61.4% for the evaluator trained on Stockfish move preferences. The test uses move comparisons, the same task the preference evaluator trains on. Against the same mobility baseline, the evaluator trained on position scores gains 254.7 Elo and the evaluator trained on move preferences 206.2. The difference between their gains is 48.5 Elo [7.9, 88.4], measured on shared openings.

Table 9: Move accuracy (%) and playing strength. The complete evaluator includes material, mobility and the integer calculations used during play.

1,285 quiet positions	Jev	Jev+Qwen
Learned evaluator	62.4	60.7
Complete evaluator	65.29	64.44
Jev+Qwen lead, Elo		+77.7
12,280 Stockfish-labelled pairs	**Preferences**	**Scores**
Learned evaluator	61.41	58.13
Complete evaluator	59.63	58.79
Gain over mobility, Elo	+206.2	+254.7

I Choosing the Weight-Size Setting

Several weight-size settings reach nearly the same validation loss, so the partner study reports two rules: each evaluator keeps its full-data setting, or each training subset selects its own. Jev+Qwen leads OLMo+Qwen and DiffusionGemma+Qwen under both (Table 3). With settings selected per subset on half the data, Jev+Qwen against Qwen trained on twice the pairs is +2.2 [−2.4, 6.8]; the full-data matches (Section 5.2) measure this comparison at +9.6 [4.3, 14.9].

We recalibrate each partner's probabilities against Stockfish, then average them with Qwen's original probabilities. The Elo changes are +3.8 [−2.1, 9.6] for OLMo+Qwen, +6.1 [−0.2, 12.4] for DiffusionGemma+Qwen and −2.1 [−8.6, 4.4] for Jev+Qwen.

J Results for Each Training Subset

Table 10 gives each training subset's results using the fixed weight-size settings.

K Examining the Othello Failure

The Othello checks examine whether the poor result arises from invalid choices or from the judgments themselves. All 127,154

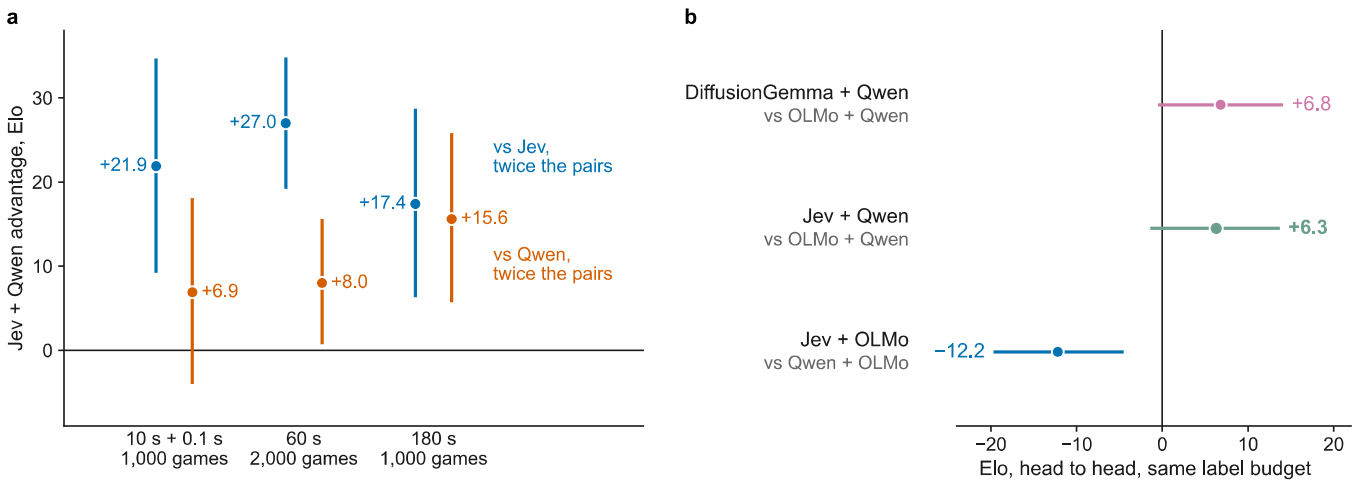


Figure 5: (a) Jev+Qwen vs. single-judge evaluators on twice the pairs at three time controls. (b) Changing one judge at equal budget, 2,000 games each. Dots: estimates; lines: 95% intervals.

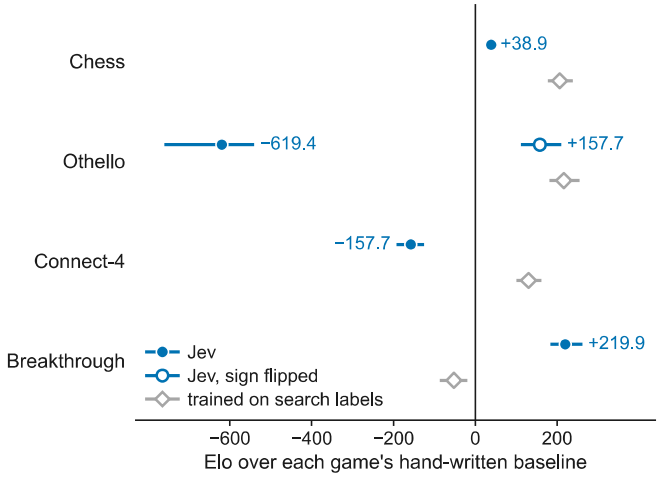


Figure 6: Playing-strength change from adding a Jev-trained score. Filled: 400-game matches (chess: 2,000; open: sign-flipped Othello, 200). Diamonds: engine-search labels. Lines: 95% intervals.

candidate moves in the Othello labels are legal for the side to move in the supplied board text. In a hand-checked position with White to move, the corner a8 captures four discs along the long diagonal; Jev assigns it probability 0.27 against the non-corner move e6. Across 5,856 pairs comparing a corner move with a non-corner move, Jev prefers the corner in 14.4% and assigns it a mean probability of 0.42. The deeper reference search labels 2,950 of these pairs and assigns the corner a mean probability of 0.69. Among the three judges tested, corner-preference rates are 17.7% for Qwen3-32B and 15.3% for OLMo; Jev's rate is 24.3% when the board is presented as a coordinate list. Training on Jev's answers produces a corner weight of −838; the baseline assigns +858. On 100 colour-paired openings, the baseline plus Jev's evaluator scores 5 wins, 2 draws and 193 losses (−603.9 Elo in this separate 200-game check, bootstrap interval [−798.3, −502.6]); the baseline minus that evaluator scores

Table 10: Elo for each evaluator against the corresponding Qwen evaluator trained on 2× the pairs, with 2,000 games per comparison. Each evaluator retains its full-data weight-size setting.

Evaluator	Subset 1	Subset 2	Subset 3
Jev+Qwen	+14.8	+7.8	+9.4
OLMo+Qwen	−10.3	−4.7	−2.8
DiffusionGemma+Qwen	−12.7	−2.3	−3.8
Qwen + Jev shuffled within Qwen groups	−1.6	+3.8	+4.2

141 wins, 3 draws and 56 losses (+157.7 Elo, [111.4, 209.9]). Doubling the baseline's corner weight gains +54.3 [13.9, 96.2]. Subtracting the full Jev-trained evaluator, including its other nine weights, gives a larger measured gain.

L Reproduction Details

Candidate moves. Candidates come from a material-only Stockfish 17.1 search (piece values 100, 300, 300, 500 and 900 centipawns, correction history off) run with MultiPV 4, 0.1 s per position and one thread. A candidate is kept when it scores within 20 centipawns of the best line; positions whose best line is a mate are dropped. The median search depth across all searched positions is 17. On the 13,000 selected positions, 72.4% have the maximum of four candidates; the mean is 3.57.

Features. Table 11 groups the 443 features.

Source games. The first block's 13,000 positions come from 2,042 games: 1,835 from the Lichess monthly database (August 2026, rated bullet and blitz, both players rated at least 2200) and 207 from the lab's own bot accounts on Lichess. Positions are sampled every fourth ply from ply 12 to 120, excluding positions in check and those with fewer than two legal moves. Test-bank positions are excluded before sampling. Of the 30,679 extracted

Table 11: The 443 features by family.

Family	Count	Examples
Piece-square	384	6 piece types $\times$ 64 squares
Mobility	8	attacked squares, safe mobility
King safety	18	zone attacks, shield pawns, castled
Pawn structure	16	doubled, isolated, passed, backward
Threats	11	hanging pieces, attacked by lower value
Other	6	bishop pair, rook on open file, outposts

positions, 21,104 have at least two candidates; 13,000 of these are selected by a fixed shuffle (seed 8).

Opening suites. Each comparison uses fresh openings drawn from a 20-ply masters opening book (Lichess opening explorer, masters and rated players above 2000). The 22-ply sets extend book leaves by two uniformly random legal plies. The 12-ply sets draw one of the book's 8-ply lines and extend it by four random plies, keeping only positions with equal material. Each generator rejects any position or move sequence already used in an earlier set; no two sets share a final position.

Bootstrap intervals. Unless otherwise stated, chess-match intervals are 95% percentile intervals from 10,000 bootstrap resamples of opening clusters (seed 20260928). The pooled headline result, the third replicate and the substitute tests cluster by single opening. Table 1 row 2 clusters by single opening (1,000 groups); rows 1 and 4 by 20-ply book prefix (556 groups); rows 3 and 5 by 8-ply prefix (411 groups).

M Prompt

Each query opens with "Position to judge:", then lists the occupied squares of each side in piece order (e.g. `White: Ke1 Qd1 Ra1 Rh1 ... e4 d4`), the side to move, and the two moves being compared in standard algebraic notation (e.g. `Candidate moves for White: Nxf6, Bd3`). The question offers the two listed moves as answer choices: "Which of these two moves is better for [White/Black]?" Both presentation orders are queried and the probabilities averaged. Jev and DiffusionGemma receive the question through the Choice interface, which requests probabilities for the specified answers. Qwen and OLMo receive a chat prompt whose allowed answers are the two moves. For the second set of labels, each Jev request presents all candidate moves for one position.

N Retrieval Details and Comparisons

We sample 80,000 passage pairs from 8,000 MS MARCO training queries, 10 pairs per query, and label each pair in both presentation orders with Jev and Qwen3-32B. Qwen receives the same passage prompt as Jev and a one-line instruction to answer A or B; its label is the probability it assigns to each answer. Each reranker initializes from MiniLM-L6-H384-uncased and trains for 10,000 steps in the main comparison and 5,000 steps in the comparisons that trade pairs for Qwen's answers (batch size 32, learning rate 2×10^{-5}) on Bradley–Terry soft labels, with 5 random seeds. The gate selects pairs by Jev's certainty $|p - 0.5|$ averaged over both orders; a pair whose two orders disagree counts as least certain. At 5,000 steps and 80,000 answers (40,000 pairs), Jev scores 0.550 and Qwen 0.560. Doubling the training steps from 5,000 to 10,000 at 160,000 answers adds $+0.038$ for Jev and $+0.030$ for Qwen.

Order consistency. On the 500 human-graded passage pairs, Jev selects the same answer in both orders on 94.8% of pairs and Qwen on 90.6%. On the 80,000 training pairs, the corresponding rates are 86.7% and 72.0%.

Combining judges at fixed pair coverage. Splitting the two presentation orders between the judges uses 80,000 answers from each and scores 0.636 in NDCG@10, a difference of -0.001 $[-0.011, 0.009]$ from Jev alone. Using both judges on every pair uses 320,000 answers and scores 0.641, a difference of $+0.004$ $[-0.003, 0.011]$, with queries resampled. These comparisons use 10,000 training steps.

Trading coverage for additional judgments. With 5,000 steps and 160,000 total answers, averaging both judges on 40,000 pairs loses 0.032 $[0.018, 0.045]$ to Jev alone on 80,000 pairs, with queries resampled. The gate asks Qwen about the 25% of pairs on which Jev is least certain. It loses 0.047 to Jev alone at 80,000 answers ($[0.013, 0.084]$) and gives a difference of -0.007 $[-0.027, 0.012]$ at 160,000 answers, with seeds and queries resampled. At 80,000 answers, dropping 20% of pairs costs 0.050, while Qwen's labels on uncertain pairs recover only 0.004.